

CSI473 Project Design Report

Campus Concierge - Team 15

Field	Complete
Assignment/phase	Phase 1 report
Approved project title	Campus Concierge
Approval decision/date	Approved with minor conditions
Team number	Group 15
Members and IDs	Thebe Segootsane 202306366; Kaelo Wadingalo 202306842; Hope Katso Toki 202307521; Katlo Ronald Benson 202304358
Repository URL	https://github.com/kaelowadingalo1-sys/CampusConcierge-
Submission tag	phase1-baseline-v1
Date	11 September 2026

1. Executive summary

Campus Concierge is a peer-to-peer campus errand platform for the University of Botswana. A requester posts an errand under an approved category, the system matches it with an available vetted helper, and both parties receive a traceable record of the task, category, standard price and deadline. The approved problem, evidence and scope are defined in Section 2.

Phase 1 limits the first vertical slice to two low-risk categories: **Parcel Collection** and **Non-assessed Document Printing/Collection**, based on six lecturer conditions (R1–R6). These are addressed through D-001 to D-007, covering admin-managed categories, prohibited errands, 10-minute acceptance and automatic reassignment, cancellation/no-show/dispute rules, no payment-data handling, controlled prototype evaluation, and a Level 0 system context.

Sections 4–7 trace **FR-01 to FR-17**, five quality scenarios, the domain model and business rules, and the errand lifecycle. The Section 6 traceability matrix links requirements to use cases, analysis elements and verification evidence, identifying items that still require written test scenarios.

Section 8 identifies the main **architecture drivers**: authoritative and auditable errand assignment; reliable 10-minute reassignment and prevention of double-assignment; contact-detail privacy; complete dispute evidence; and graceful handling of helper backlogs. Key constraints include no payment processing/storage, a mocked identity service, a closed admin-managed category list, and delivery by a small student team within one semester. Phase 2 will use these drivers to select architectural views, styles, alternatives and ADRs.

2. Problem evidence, stakeholders and scope

2.1 Problem statement

Students at the University of Botswana regularly need small errands run: collecting a parcel from a courier point, standing in a long registration or bank queue, buying an item in town, or printing and submitting a document before a deadline. Lectures, labs and part-time commitments often make it impossible for a student to do this themselves in time. Students currently ask around informally, usually in residence WhatsApp groups or by word of mouth, and this works unevenly: a request can sit unanswered for hours, a willing helper is not always someone the requester knows or trusts, and once someone agrees there is no shared record of what was actually agreed, including the task, the price and the deadline. When something goes wrong, such as a helper who does not show up, buys the wrong item, or a disagreement over whether payment was made, neither side has anything to point back to. Campus Concierge addresses this by giving students one shared place to post an errand, get automatically matched with a vetted, registered helper, and keep a traceable record of the deal as it happens.

2.2 Evidence of the problem

Evidence/source	Date/origin	What it shows
Informal survey of 12 students across two residence WhatsApp groups	Aug 2026	9 of 12 respondents said they had asked around for help with an errand at least once this semester; several reported requests going unanswered for over an hour
Short interviews with 4 students who had informally helped with or requested an errand	Aug 2026	All 4 said there was no shared record of what was agreed; 2 described a disagreement afterward over price or timing

2.3 Approval conditions addressed

The proposal was approved with minor conditions (six conditions, R1-R6). Each is addressed by a design record in the repository:

Ref	Lecturer's condition	Addressed by
R1	Restrict the first vertical slice to one or two low-risk campus errand categories.	D-001

Ref	Lecturer's condition	Addressed by
R2	Prohibit errands involving bank/identity representation, assessed-work submission, confidential documents, restricted goods, high-value purchases, unsafe off-campus travel, or impersonating the requester.	D-002
R3	Define helper acceptance timeouts, reassignment, cancellation, requester no-show, helper no-show, and dispute-resolution rules.	D-003, D-004
R4	Do not process or store payment information.	D-005
R5	Evaluate matching using controlled prototype scenarios; do not run a real pilot unless formal permission is obtained.	D-006, prototype/scenarios/, tests/matching-scenario-results.md
R6	Submit a Level 0 system context diagram showing requester, helper, administrator, and relevant notification/identity services.	models/context/level-0-context.drawio (Figure 2.1)

2.4 Level 0 system context

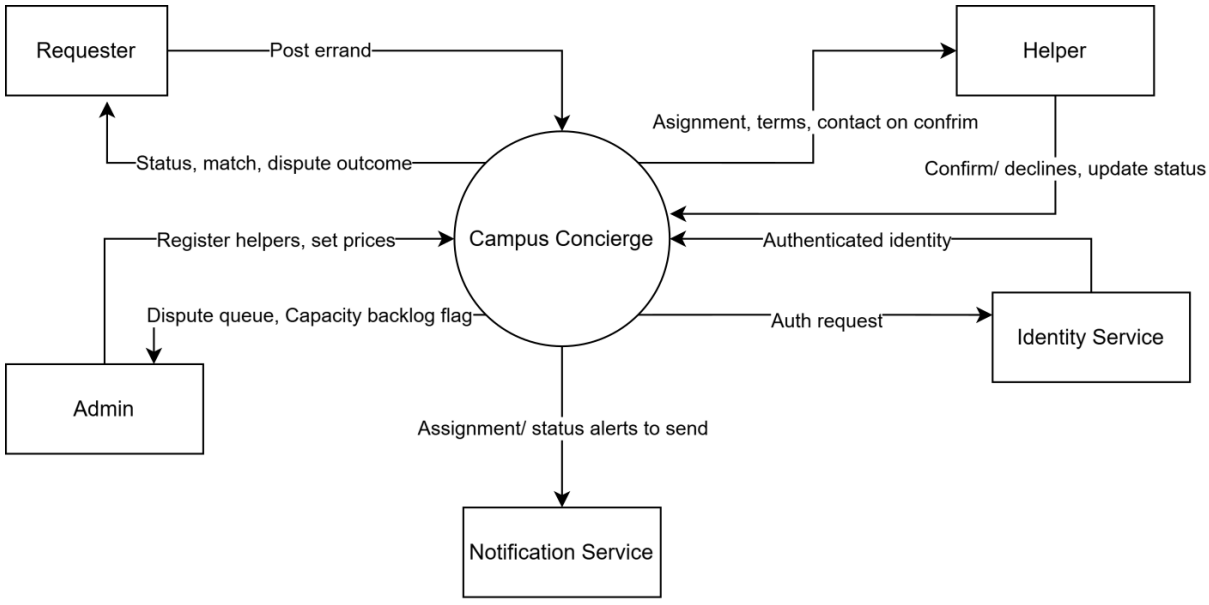


Figure 2.1: Level 0 system context diagram (models/context/level-0-context.drawio).

2.5 Goal and objectives

Goal: enable students to find a trustworthy peer helper for a campus errand and keep a shared, traceable record of the agreed deal, so disputes and unresolved no-shows become rare rather than routine.

- Objective 1: in controlled prototype test scenarios, correctly match at least 80% of scenario cases with an eligible helper within the 10-minute acceptance window, without a real pilot (per D-006).
- Objective 2: ensure at least 90% of matched deals have an explicit, timestamped record of task category, standard price and deadline stored before the helper marks work as started.
- Objective 3: provide a dispute-report workflow that captures enough evidence to reconstruct at least 90% of test dispute scenarios without relying on external chat logs.

2.6 Stakeholders

Stakeholder/role	Need	Concern	Evidence/access
Requester (student needing an errand run)	Find a willing helper quickly and trust the deal will be honoured	Helper doesn't show, or terms are disputed after the fact	Team members are themselves potential requesters; accessible via peer interviews
Helper (vetted student registered to run errands)	Get auto-assigned errands and clear task terms	Unfairly blamed with no record of what was actually agreed	Accessible via peer interviews with students who have informally helped others
App admin (helper vetting and assignment)	A reliable pool of helpers, and standard prices per category that stay fair	Not enough available vetted helpers, or a category price that drifts out of step with real effort	Team's own operational role running the vetting and registration process
Residence/hall administration	Deliveries and meetups don't disrupt hall or accommodation operations	Unknown visitors arriving at hall entrances for handoffs	Informal conversation with a hall porter about visitor handoffs at the entrance
University IT	A real identity system would need student accounts to be genuinely trustworthy	The prototype's mocked login (D-007) does not prove real integration with UB systems would work	No direct engagement this semester (D-007); UB's Microsoft 365 use confirmed only via UB's public site

2.7 Scope

In scope

- Posting an errand request under a defined category with its standard price
- Registering students as helpers, and auto-assigning a posted errand to an available registered helper
- Recording task details, category and deadline against the category's standard price
- Errand status lifecycle and dispute reporting

Out of scope

- Real payment processing or holding funds
- Per-errand price negotiation between requester and helper
- Open self-signup as a helper without admin approval
- University disciplinary adjudication
- Background or criminal-record checks beyond the team's own vetting

2.8 Assumptions and constraints

Assumptions

- Students authenticate with their existing university student number/email
- Cash or mobile money changes hands outside the app
- New helpers are recruited and approved by the admin team, not by open self-signup
- Each errand category has a fixed standard price set by admins, not negotiated per errand

Constraints

- Must remain usable on limited campus data or wifi
- Must be deployable and supportable by a small student team
- Errand capacity is limited by how many vetted helpers are currently registered
- Delivered within one semester as a vertical slice, not a full product

3. Glossary

Terms used consistently across the problem statement, stakeholder table and all downstream artefacts.

Term	Working definition
Requester	A student who posts an errand they need completed.
Helper	A student who has been vetted and registered by an administrator to complete errands.
Errand	A request in a defined category with task details, a deadline and a standard price.
Errand category	An administrator-managed grouping of errands that has one fixed standard price.
Deal	The traceable agreement created when a helper confirms an assigned errand.
Standard price	The non-negotiable price recorded for an errand category; the system records it but does not process payment.
Assignment	The system's allocation of a posted errand to the next available registered helper.
Reassignment	Allocation to another eligible helper after a decline or missed response window.
Backlog	The state of a posted errand with no eligible helper currently available; retried as helpers free up.
Dispute	A requester or helper report that an errand was not completed as agreed, supported by the recorded deal.
App admin	The role that vets helpers, maintains category prices and monitors assignment capacity.
Identity Service	The external entity a real login would depend on; mocked for this semester's prototype (D-007).
Notification Service	The external channel used for signup confirmation and assignment/status/dispute alerts.
Errand status	The lifecycle a single errand moves through: POSTED, AWAITING_ACCEPTANCE, ACCEPTED, IN_PROGRESS, COMPLETED, CLOSED, with exception paths to REASSIGN, CANCELLED or DISPUTED (Figure 7.1).

4. Functional requirements and use cases

4.1 Actors

- Requester - a student who posts an errand
- Helper - a vetted, registered student who is assigned errands
- Admin - vets helpers, sets category prices, resolves disputes
- Notification Service (supporting, external) - delivers assignment/status/dispute alerts
- Identity Service (supporting, external) - authenticates users

4.2 Use-case diagram

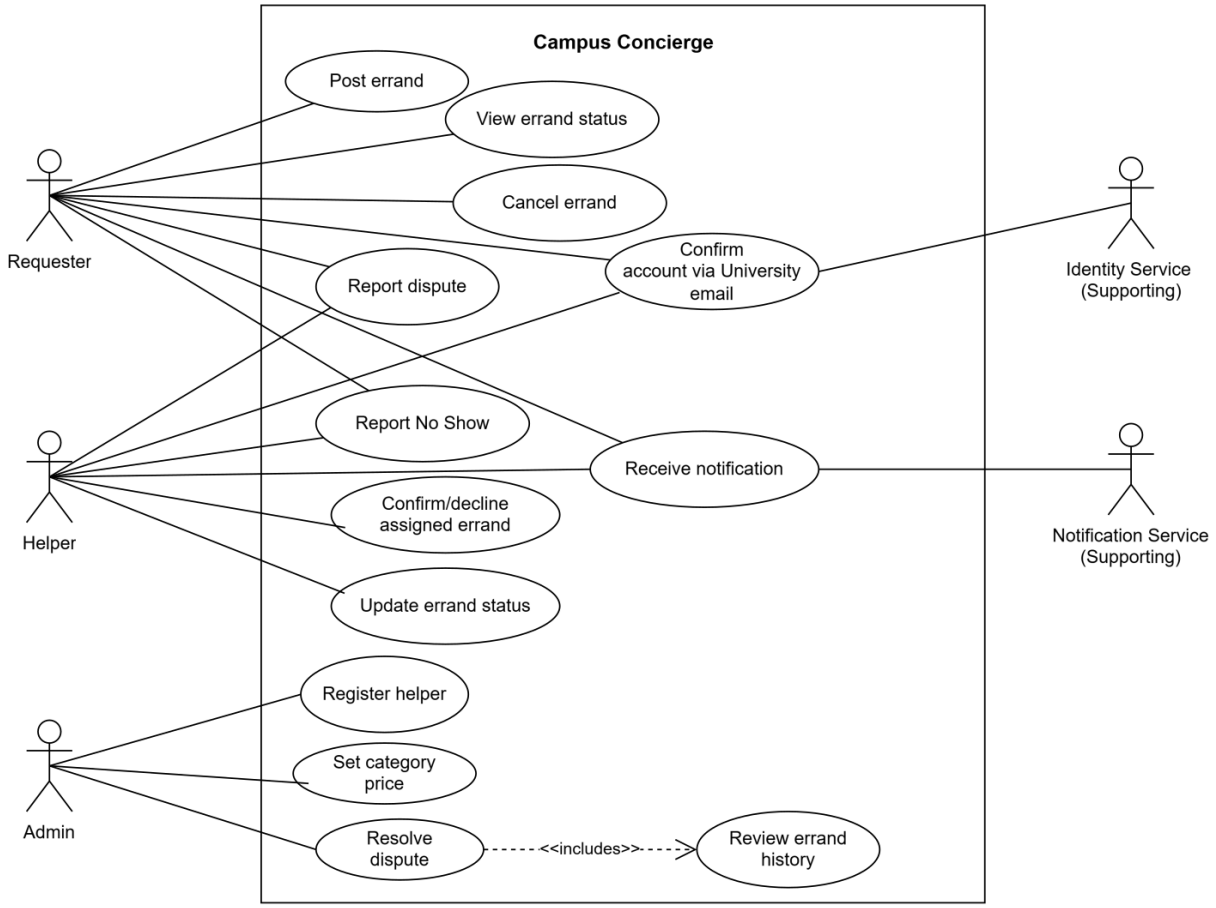


Figure 4.1: Campus Concierge use-case diagram (models/use-cases/campus-concierge-use-cases.drawio).

4.3 Functional requirements

Each requirement is written as an actor goal and traces to the design-record decision that justifies it. This list covers the first vertical slice only (Parcel Collection and Non-assessed Document Printing/Collection, per D-001).

ID	Requirement	Traces to
FR-01	A requester can post an errand request under an approved category, with task details and a deadline.	D-001, D-002
FR-02	The system automatically assigns a posted errand to an available registered helper.	D-003
FR-03	An assigned helper can confirm or decline the errand within the acceptance window.	D-003
FR-04	If a helper declines or does not respond within the acceptance window, the system reassigns the errand to the next eligible helper.	D-003
FR-05	Once an errand is confirmed, the system locks the agreed terms (task, category, standard price, deadline) and reveals contact/meeting details to both parties.	D-003, D-005
FR-06	A requester can cancel an errand freely before acceptance; cancellation after acceptance requires a stated reason and is recorded.	D-004
FR-07	A helper can report a requester no-show after waiting the defined window at the handoff point.	D-004

ID	Requirement	Traces to
FR-08	A requester can report a helper no-show if the helper does not appear within the agreed window.	D-004
FR-09	Either party can open a dispute on a contested or failed errand; the system captures the full recorded event history as evidence.	D-004
FR-10	An administrator can review a dispute's recorded evidence and resolve it with a retained, timestamped outcome.	D-004
FR-11	An administrator can vet and register a student as a helper.	D-002 (App admin stakeholder)
FR-12	An administrator can set and adjust the standard price for an errand category.	Category pricing decision
FR-13	The system rejects an errand posted under a prohibited or non-approved category at creation, not later.	D-001, D-002 (BR-01, BR-02)
FR-14	The system never requires a helper to perform financial, identity, academic-assessment, or impersonation representation, regardless of category.	D-002 (BR-03)
FR-15	The system does not process, transfer, or store payment information; only the category's standard price is recorded as data.	D-005
FR-16	A student creates an account by confirming a link sent to their university email address; the account is self-service after that.	D-007
FR-17	A requester or helper receives status, assignment, and dispute notifications through the Notification Service.	Level 0 context diagram

4.4 Detailed use case: Post Errand Request

This is the use case worked through in full detail; it anchors the acceptance criteria (4.5) and the interaction walkthrough in Section 7.

Actor: Requester

Precondition: The requester has a confirmed account (D-007).

Postcondition: A validated errand exists in POSTED status, tied to an approved category and its current standard price, with the requester able to see its status.

Main success scenario

- 1. Requester selects an approved errand category.
- 2. Campus Concierge retrieves the category's current standard price and general backlog status.
- 3. Requester supplies the task details and a deadline.
- 4. Campus Concierge validates the category against the approved/prohibited list (BR-01, BR-02).
- 5. Campus Concierge records the errand as POSTED, with the category's standard price locked in as the recorded price.
- 6. Campus Concierge confirms the errand identifier and status to the requester.

Alternative flows

Scenario	Required system response	Design principle
Category is not on the approved list	Reject the request before creating any errand record.	Access policy (BR-01, BR-02 / D-001, D-002)
No eligible helper is currently available	Still record the errand as POSTED; flag the capacity backlog to the Administrator.	Failure policy (D-003)
Requester's account is not yet confirmed	Refuse posting until university-email confirmation completes (D-007).	Access policy
Duplicate submission (same task resubmitted quickly)	Return the existing pending errand rather than creating a second one.	Idempotency
Category's standard price changed between the requester viewing it and submitting	Use the current standard price at the moment of submission, not a cached one from step 2.	Version identity

4.5 Acceptance criteria

ID	Given	When	Then
AC-1	An approved category, complete task details and a deadline	The requester submits the errand	It is recorded as POSTED with the category's current standard price locked in
AC-2	A category not on the approved list (D-002)	The requester attempts to submit	The system rejects it before any errand record is created
AC-3	No eligible registered helper is currently available	An errand is posted	It stays POSTED, enters the backlog, and a capacity flag is raised to the administrator
AC-4	An assigned helper has not responded	10 minutes elapse (D-003)	The errand reassigns to the next eligible helper and the timeout is recorded against the original helper
AC-5	The requester's account is not yet confirmed via university email (D-007)	They attempt to post an errand	The system refuses posting until confirmation completes

5. Quality requirements

Five quality scenarios, each a distinct quality attribute, written in stimulus/context/response/measure form, and each traced to the design record and (where one exists yet) the controlled prototype scenario that evaluates it.

#	Attribute	Stimulus	Context	Response	Measure
QS-1	Reliability	Assigned helper does not accept or decline	Helper has one active assignment; device may be offline or silent	Assignment expires; errand reassigns to the next eligible helper; timeout recorded against the original helper (D-003)	Reassignment within 10 minutes; no errand left in AWAITING_ACCEPTANCE beyond that window
QS-2	Privacy	An errand is posted and awaiting a match	Other students can browse open errands	Contact and meeting details stay hidden until a helper's assignment	Zero contact-detail exposure to any party before confirmation, checked on every posted errand

#	Attribute	Stimulus	Context	Response	Measure
				is confirmed by both parties	
QS-3	Availability	An errand is posted with zero eligible helpers currently free	Normal operation, small registered helper pool	Errand enters a backlog state; a capacity flag is raised to the administrator; assignment retries as helpers free up	Backlog flag raised within the same request cycle that found zero helpers; no errand silently dropped
QS-4	Auditability	A dispute is opened on a contested errand	Either party can open a dispute at any point after acceptance	The system reconstructs the full event history (assignment, confirmation, status changes, timestamps) for admin review	100% of recorded fields available to the admin; no dispute reviewed with missing event history
QS-5	Scalability	Several errands are posted at close to the same time	Helper pool smaller than the number of posted errands	Errands are assigned in a defined order (e.g. post-time); the remainder enter backlog; no helper receives two active assignments	Zero double-assignments across any competing batch, regardless of batch size

5.1 Resulting design obligations

Each scenario places a concrete obligation on the design, independent of which architecture eventually satisfies it:

- QS-1 -> D-003; evaluated by prototype/scenarios/helper-timeout.md. Obligation: the system must be able to expire an assignment and reassign without manual intervention.
- QS-2 -> D-004, D-007. Obligation: contact/meeting detail must not be readable through any path before both-party confirmation.
- QS-3 -> D-003; evaluated by prototype/scenarios/no-helper-available.md. Obligation: 'no eligible helper' must be a first-class, recorded state, not a silently dropped request.
- QS-4 -> D-004, D-007. Obligation: every status change, assignment attempt and timestamp on an errand must be retained and retrievable as a single, complete history.
- QS-5 -> evaluated by prototype/scenarios/multiple-errands-compete.md. Obligation: assignment must be atomic enough that two concurrent posts can never both capture the same helper.

6. Analysis/domain model and responsibilities

6.1 Domain model

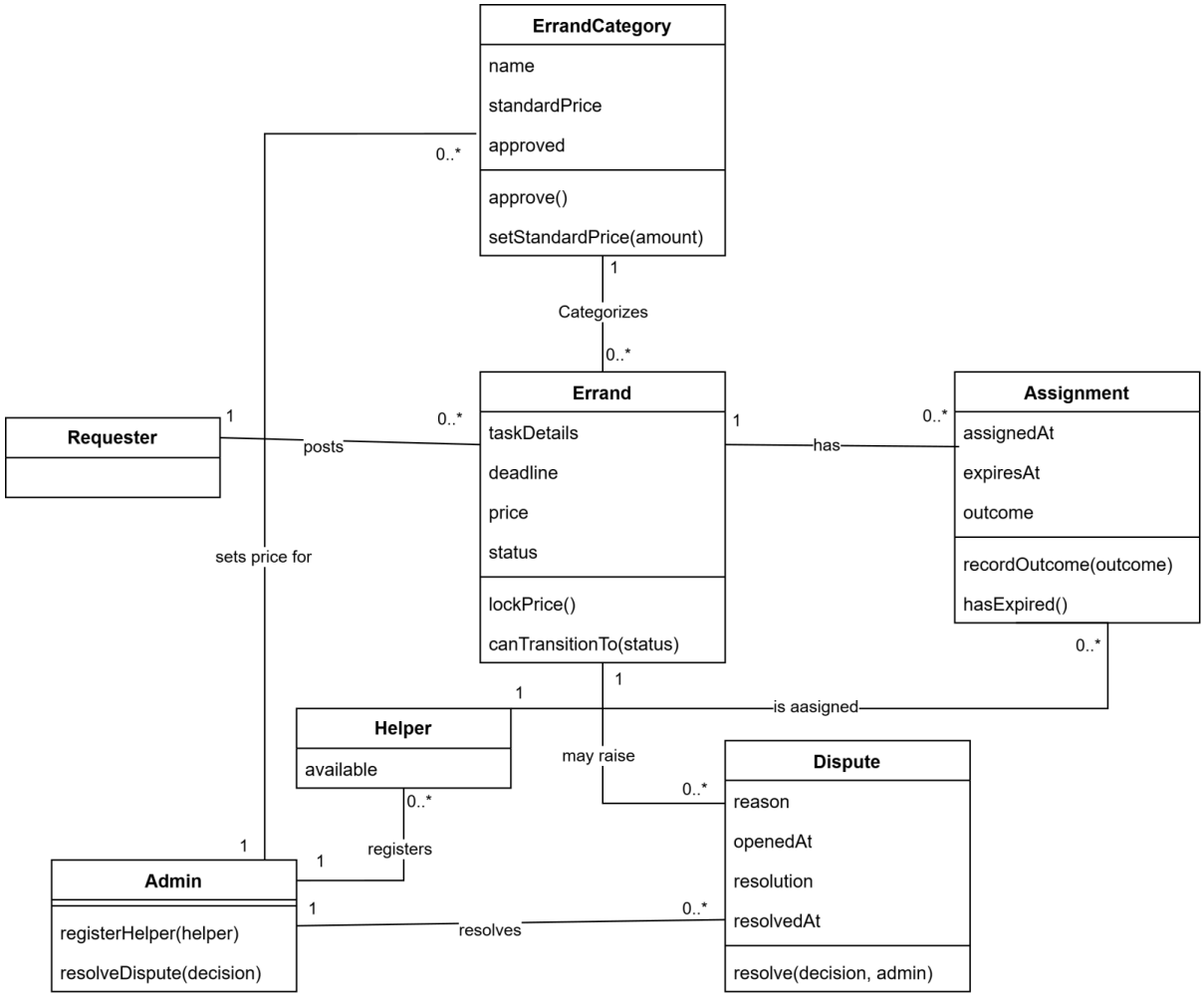


Figure 6.1: Campus Concierge domain class model (models/domain-model.drawio).

Two scope decisions were made while building this model (D-008, status: proposed):

- Requester, Helper and Administrator are separate classes, not roles of one User class, because a single User-with-role class would carry Helper-only state (available) and Administrator-only behaviour (setting prices, resolving disputes) as optional fields most users never populate.
- Assignment is its own persistent class, not a field on Errand, because an errand can accumulate a history of assignment attempts (decline, timeout, reassignment, per D-003), which a single currentHelperId field cannot represent, and D-004 needs that full history as dispute evidence.

6.2 Business rules

Domain invariants (enforced by the domain model itself)

- BR-D1: An Errand's category must be approved at the moment the Errand is created (D-002, BR-01/BR-02).
- BR-D2: An Errand's price is fixed to its category's standard price at post time and does not change even if the category's price changes later (D-001, pricing decision).
- BR-D3: A Helper can be the subject of at most one active (non-terminal) Assignment at a time (D-003, no double-assignment).
- BR-D4: An Assignment's outcome, once recorded, is immutable (supports audit and dispute evidence, D-004).
- BR-D5: An Errand may have at most one Dispute, since DISPUTED is a terminal lifecycle state (D-004).
- BR-D6: Only an Administrator may change an ErrandCategory's approved flag or standard price (D-002, pricing decision).
- BR-D7: A Dispute's resolution can only be set by an Administrator, never directly by the Requester or Helper (D-004).

Categories (D-001, D-002)

- Only two errand categories are enabled in the first vertical slice: Parcel Collection and Non-assessed Document Printing/Collection.
- Prohibited categories, at any stage: bank/identity representation, submission of assessed academic work, confidential/sensitive documents, restricted goods, high-value purchases, unsafe off-campus travel, or anything requiring the helper to impersonate the requester.
- BR-01: only enabled, admin-approved categories may be posted.
- BR-02: a prohibited-category errand is rejected at creation, not caught later.
- BR-03: a helper is never asked for financial, identity, academic-assessment, or impersonation representation.
- BR-04: the first slice's UI and API only expose the two approved categories.

Assignment and timeouts (D-003)

- A helper has 10 minutes to accept or decline an assigned errand.
- Decline or timeout leads to immediate reassignment to the next eligible helper.
- Every declined/expired assignment is recorded against the original helper.
- No eligible helper available leads to the errand entering a backlog state and retrying as helpers free up.

Cancellation and no-shows (D-004)

- Requester can cancel freely before a helper accepts; after acceptance, cancellation needs a stated reason and is recorded.
- Helper waits 10 minutes at the handoff point before reporting a requester no-show.
- Requester reports a helper no-show if the helper doesn't appear in the agreed window; either no-show can be escalated to a dispute.

Disputes (D-004)

- Either party can open a dispute on a contested or failed errand; the system captures the full recorded event history as evidence.
- An app admin reviews and resolves the dispute; the resolution is final within this system (no automated or legal adjudication).

Payment (D-005)

- No payment is processed, transferred, or held by the system, and no payment credentials are ever stored.
- Only the category's fixed standard price is recorded as data, as part of the deal.

Evaluation and identity (D-006, D-007)

- The matching/assignment logic is evaluated against controlled, synthetic scenarios; no real pilot runs without separate formal permission.
- The identity service is mocked for the prototype rather than integrated with a real system, since D-006 already rules out a real pilot.

6.3 CRC cards

Class	Responsibilities	Collaborators
Errand	Know its own task details, category, price, deadline and status Validate that a requested status transition is legal Lock its price to the category's standard price at posting time (BR-D2)	ErrandCategory, Assignment, Dispute, Requester
Assignment	Know which helper it was offered to, and when Record its own outcome once decided: accepted, declined, or timed out Know whether its acceptance window has expired	Errand, Helper
ErrandCategory	Know its own approved status and standard price	Errand, Administrator

Class	Responsibilities	Collaborators
	Refuse to let a new Errand reference it while not approved (BR-D1)	
Administrator	Vet and register a Helper Set an ErrandCategory's approved flag and standard price Review a Dispute's evidence and resolve it	Helper, ErrandCategory, Dispute
Dispute	Know its reason, the Errand it concerns, and when it was opened Record its resolution and when it was resolved Refuse a resolution set by anyone other than an Administrator (BR-D7)	Errand, Administrator

6.4 Traceability matrix

Requirement -> use case -> analysis element -> verification.

Requirement	Use case	Analysis element	Verification
FR-01	Post Errand Request	Errand, ErrandCategory	AC-1, AC-2
FR-02	Post Errand Request	Assignment	prototype/scenarios/multiple-helpers-available.md
FR-03	Confirm or Decline Assigned Errand	Assignment	prototype/scenarios/helper-accepts.md
FR-04	Confirm or Decline Assigned Errand	Assignment	prototype/scenarios/helper-declines.md, helper-timeout.md
FR-05	Confirm or Decline Assigned Errand	Errand, Assignment	AC-1
FR-06	Cancel Errand	Errand	Pending, no scenario yet
FR-07	Report No Show	Dispute	Pending, no scenario yet
FR-08	Report No Show	Dispute	Pending, no scenario yet
FR-09	Report Dispute	Dispute, Assignment, Errand	QS-4
FR-10	Resolve Dispute	Administrator, Dispute	QS-4
FR-11	Register Helper	Administrator, Helper	Pending, no scenario yet
FR-12	Set Category Price	Administrator, ErrandCategory	Pending, no scenario yet
FR-13	Post Errand Request	ErrandCategory, Errand	AC-2
FR-14	(policy, no single use case)	ErrandCategory	Pending, no scenario yet
FR-15	(policy, no single use case)	Errand, price attribute only	Pending, no scenario yet
FR-16	Confirm Account via University Email	outside domain model, see D-007	Pending, no scenario yet
FR-17	Receive Notification	outside domain model, external service	Pending, no scenario yet

7. Interaction and lifecycle

7.1 Errand lifecycle (state model)

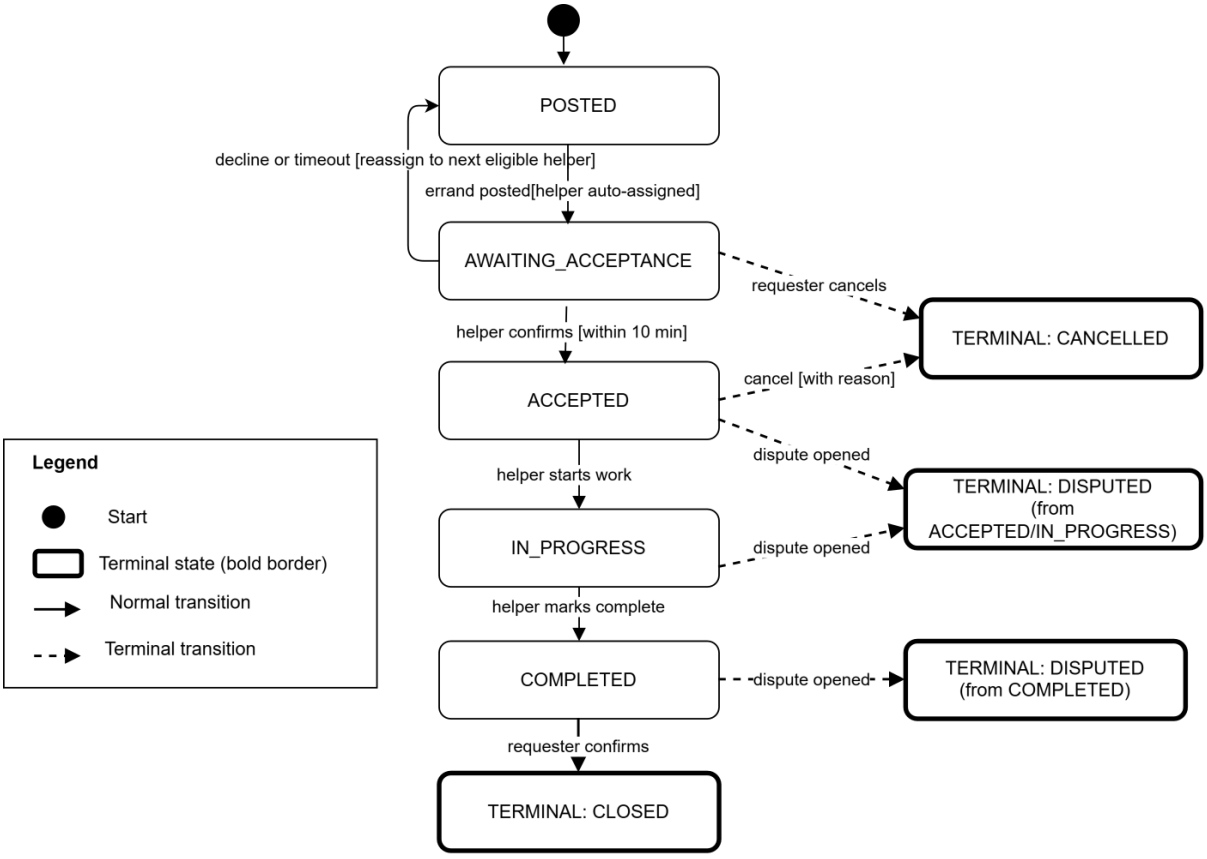


Figure 7.1: Errand lifecycle state diagram (models/state/errand-lifecycle.drawio).

An Errand starts at POSTED. Once a helper is auto-assigned it moves to AWAITING_ACCEPTANCE; the helper's confirmation within the 10-minute window (D-003) moves it to ACCEPTED, while a decline or timeout returns it to POSTED for reassignment to the next eligible helper. From ACCEPTED the helper starts work (IN_PROGRESS) and later marks it complete (COMPLETED); the requester's confirmation closes it (terminal CLOSED). CANCELLED is terminal and reachable only from AWAITING_ACCEPTANCE (free cancellation) or ACCEPTED (cancellation with a stated reason, D-004). DISPUTED is terminal and reachable from ACCEPTED, IN_PROGRESS or COMPLETED, reflecting that either party may open a dispute at any point after acceptance (D-004, QS-4).

7.2 Interaction: Post Errand Request

Figure 7.2 formalises the main flow first described as a numbered walkthrough in the Phase 1 draft. The requester posts the errand; Errand checks and locks in the category's approved status and standard price (BR-D1, BR-D2); Errand then creates the first Assignment and the Assignment notifies the helper through the Notification Service. The alt frame captures the two branches AWAITING_ACCEPTANCE can take (Section 7.1): the top compartment is the helper confirming within the window, which locks the terms and reveals contact details to both parties (AD-05); the bottom compartment is a decline or an expired window, which records that outcome on the Assignment and requests reassignment to the next eligible helper (AD-02).

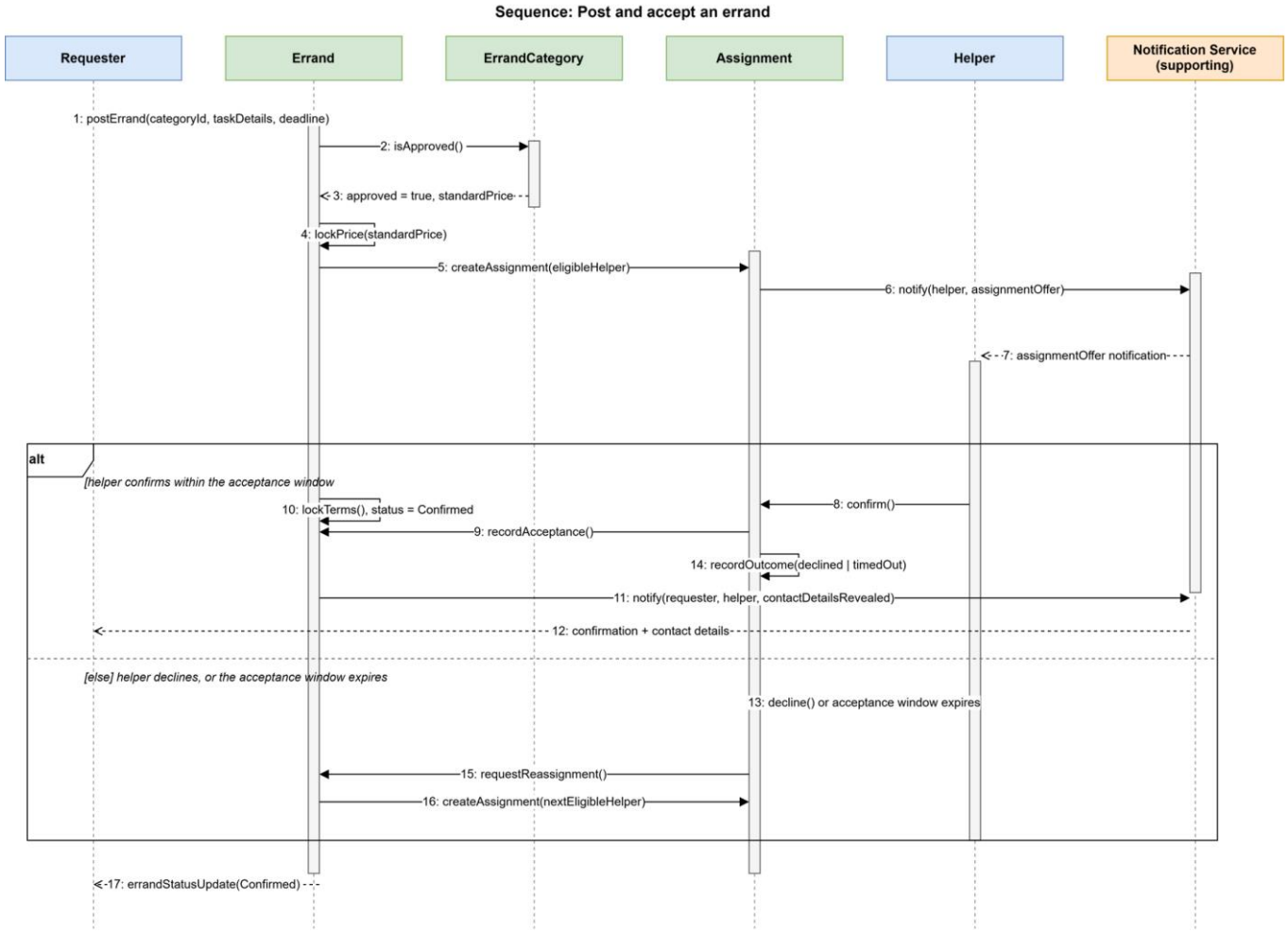


Figure 7.2: Sequence diagram for Post Errand Request - main flow and the confirm/decline-timeout alternative

7.3 Consistency across models

The status values named in the domain model's Errand.status attribute (Section 6.1) are exactly the states drawn in Figure 7.1: POSTED, AWAITING_ACCEPTANCE, ACCEPTED, IN_PROGRESS, COMPLETED, CLOSED, CANCELLED and DISPUTED. Errand.canTransitionTo(status) is the operation responsible for enforcing that only the transitions drawn in Figure 7.1 are legal. The Assignment class's assignedAt, expiresAt and outcome attributes are what the AWAITING_ACCEPTANCE state and its decline/timeout transition are backed by, and Assignment.hasExpired() is the operation the QS-1 reassignment obligation (Section 5.1) depends on. Figure 7.2's messages 4, 10 and 14 (lockPrice, lockTerms, recordOutcome) are exactly those operations; message 14's declined/timedOut outcome and message 15's requestReassignment() are what drive the AWAITING_ACCEPTANCE -> POSTED transition back in Figure 7.1.

.

8. Architecture and decisions

This Phase 1 baseline covers architecture drivers only. Alternatives, the component model, ADRs and their consequences are Phase 2 work and are left as placeholders at the end of this section.

8.1 Architecture drivers identified during the Phase 1 review

Drivers were pulled from three sources, following the method used in class: functional responsibility, quality scenario, and constraint/dependency, then each was filtered by essence (is it central to the system?), meaning (what does it mean here?) and fit (what does it affect?).

ID	Source	Driver	Essence / meaning / fit
AD-01	Functional responsibility (FR-02 to FR-05)	Own the authoritative assignment of a posted errand to an eligible helper, including its full attempt history.	Essence: central, this is the system's reason to exist. Meaning: assignment state

ID	Source	Driver	Essence / meaning / fit
			and history must be a first-class, queryable record, not a log line. Fit: affects Errand, Assignment, dispute evidence (QS-4), and any future matching logic.
AD-02	Quality scenario QS-1 (reliability)	A non-responding helper's assignment must expire and reassign within 10 minutes, unattended.	Essence: central, directly protects the requester's deadline. Meaning: expiry is a timed state transition, not a manual admin action. Fit: affects Assignment's lifecycle and whatever component owns time-based transitions.
AD-03	Quality scenario QS-5 (scalability under concurrency)	Two errands posted close together must never both capture the same helper.	Essence: central, a double-assignment breaks BR-D3 and the trust the whole system depends on. Meaning: assignment must be an atomic operation, not read-then-write. Fit: affects however Helper availability and Assignment creation are coordinated.
AD-04	Quality scenario QS-3 (availability with zero helpers)	A posted errand with no eligible helper must become a visible backlog entry, never a silently dropped request.	Essence: central to trust; a dropped errand is worse than a slow one. Meaning: 'no helper available' is a state, not an error path that terminates the request. Fit: affects Errand's state model and whatever notifies the Administrator of capacity backlog (context diagram, Figure 2.1).
AD-05	Quality scenario QS-2 (privacy)	Contact and meeting details must stay hidden until both parties have confirmed the assignment.	Essence: not central to the core workflow, but a hard safety/trust boundary. Meaning: this is an access-control rule that must hold regardless of which interface renders the errand. Fit: affects every read path that can expose an Errand's contact/meeting fields.
AD-06	Quality scenario QS-4 (auditability)	The full event history of an errand (assignment, confirmation, status changes, timestamps) must be reconstructable for a dispute review.	Essence: central, this is what makes the dispute workflow (FR-09, FR-10) possible at all. Meaning: history must be retained, not just current state. Fit: affects Assignment (BR-D4, immutable outcome) and however Errand's status history is stored.

ID	Source	Driver	Essence / meaning / fit
AD-07	Constraint/dependency (D-005)	The system must never process, transfer, or store payment information; only a category's fixed standard price is recorded as data.	Essence: not central to matching, but a non-negotiable boundary set by condition R4. Meaning: no design may introduce a payment-credential field or a payment-processing call, however convenient. Fit: affects ErrandCategory.standardPrice and rules out an entire category of external integration.
AD-08	Constraint/dependency (D-007, R6)	A real identity system is out of scope this semester; the Identity Service is mocked, but stays on the system boundary as a named external entity.	Essence: not central to this semester's evaluation (D-006), but shapes where authentication sits. Meaning: authentication must be swappable behind a stable boundary, since the mock is explicitly provisional. Fit: affects how Requester/Helper/Admin identity is established without coupling the rest of the system to the mock's implementation.
AD-09	Constraint/dependency (D-001, D-002)	The errand category list is closed and admin-managed; every errand-creation path must validate against it, not just the UI.	Essence: central to safety (R1, R2); this is a lecturer-imposed guardrail, not a style choice. Meaning: category validation is policy that must be enforced wherever an Errand can be created, not decoration on one screen. Fit: affects ErrandCategory, BR-01/BR-02 enforcement, and any additional entry point added later (e.g. an API).
AD-10	Constraint/dependency (project scope)	The system must be built, run and supported by a small student team within one semester, evaluated only against controlled scenarios (D-006), not a live pilot.	Essence: not part of the problem domain, but bounds every other decision's cost. Meaning: favour the simplest structure that satisfies AD-01 to AD-09 over one that anticipates scale or users this project will not have. Fit: affects the choice of architecture style and deployment granularity to be made in Phase 2.

8.2 Alternatives, component model, ADRs and consequences

[Alternatives; component model; ADRs; consequences. To be completed for the Phase 2 submission, working from the drivers in Section 8.1.]

9. Data, API/service and deployment design

[Logical data model; contracts; deployment and relevant failure/recovery behaviour.]

10. Detailed design, patterns and framework

[Interfaces/classes; patterns; rejected alternatives; framework boundary.]

11. User-interface, accessibility and usability

[Task flows; wireframes; alternative states; accessibility and usability evidence.]

12. Implementation vertical slice

[Approved core use case; repository structure; setup and run instructions.]

13. Testing and results

[Requirement-linked success, rule/validation and failure tests; quality evidence.]

14. Reconciliation and change history

[Model-to-code-to-test mapping; differences; artefacts updated.]

15. Risks, technical debt and release recommendation

[Residual risks; priorities; conditions for further use.]

16. Academic integrity and AI-use record

[Summarise disclosed assistance and verification.]

References

[Use a consistent citation style.]

Appendices

[Approval form; traceability matrix; ADRs; contribution record; detailed test logs as needed.]